

Structural Retrieval versus Similarity Search: A Measured Comparison on SEC EDGAR Filings

Fakabbir Amin

fakabbir@gmail.com

September 2026

Abstract

Enterprise document corpora routinely store facts about the same entity in different document types, written by different teams at different times. We ask whether a knowledge graph keyed on stable entity identifiers retrieves such facts better than similarity search over embeddings, and we measure the answer rather than assert it. Our corpus is one snapshot of SEC EDGAR [1]: 167,050 filings from 10,511 companies, 508,714 embedded chunks of risk-factor text, and a graph of 430,257 nodes and 561,144 edges carrying an accession number on every edge. Three architectures — text-to-SQL over typed columns, dense-vector RAG with HNSW cosine search [2, 3], and a two-stage GraphRAG design — share one model, temperature and token budget, and are scored by a programmatic judge over 20 questions in four categories, each asked three times, for 180 scored attempts. GraphRAG solved 16 of 20 questions on all three attempts to text-to-SQL’s 10 and vector RAG’s 3, winning three categories and losing structured aggregation as predicted, 6 of 15 attempts against 15 of 15 at 3.6 times the token cost. Vector RAG lost the single-document category it was expected to win, passing 3 of 15 where GraphRAG passed 15 of 15, because “belongs to company *X*” is not recoverable from cosine similarity at this scale. We also report what an earlier version of this measurement got wrong: a name-normalisation convention was undocumented in both schema descriptions, which we had published as a category loss for the graph. Documenting it identically for all arms, and changing nothing else, moved three of four categories — and raised text-to-SQL’s confident falsehoods from 2 to 6, because better schema knowledge converted refusals into assertive multi-hop answers that conflate people sharing a surname. GraphRAG made no false statement in either measurement. We claim no advantage beyond this corpus, this model and this question set.

Keywords: retrieval-augmented generation, knowledge graphs, text-to-SQL, entity resolution, provenance, SEC EDGAR, empirical evaluation.

1 Introduction

Retrieval-augmented generation has become the default way to put a language model in front of a document corpus [2, 4]. The standard recipe embeds passages, retrieves the nearest neighbours of an embedded question, and lets the model answer from what came back. It works well when the answer is contained in a passage that resembles the question.

A large class of enterprise questions is not like that. Consider a question a financial analyst would actually ask: *for every company where a particular person is an officer or director, list that company’s disclosed subsidiaries and summarize the cybersecurity risks it reported.* Three facts are required. The person’s role is disclosed in an insider-ownership filing. The subsidiaries are disclosed in an exhibit attached to the annual report. The risks are narrative prose inside a numbered section of that same report. These are three separate documents, in three formats, produced by three different processes.

The difficulty is not that the passages are hard to rank. It is that no passage contains the combination. We measured this directly on our corpus: the surname of the subject of our hardest question appears in **0 of 5,210** extracted risk-factor sections, and only **1 of 151** subsidiary names appears anywhere in its own parent’s risk-factor section. Ownership and risk are, empirically, disjoint. Figure 1 shows the structure that produces this.

This has a consequence that is easy to state and easy to overlook: a better encoder cannot fix it, and neither can a longer context window. Retrieval can only return text that exists. If the conjunction of facts a question needs is never co-located in any passage, then ranking passages more accurately retrieves the same insufficient evidence. The ceiling is a property of the corpus, not of the retriever.

There is a second and subtler failure. Similarity search has no mechanism for filtering by entity, because “belongs to company X ” is not a semantic property of a passage. Two risk-factor sections about supply-chain disruption are near each other in embedding space whether or not they belong to the company being asked about. At small scale this rarely bites, because few candidates compete. We show that it bites hard at 508,714 chunks drawn from 5,210 filers, and — more usefully — that it bites on exactly the questions where similarity search is supposed to be the right tool.

The obvious alternative is to store the structure explicitly: resolve entities to stable identifiers, materialize the relationships as edges, and attach a provenance key to every edge so that any claim can be traced to the document that supports it. This is the family of designs now loosely called GraphRAG [5, 6]. The question this paper addresses is not whether that is possible but whether it is *worth it*, and on which question shapes — measured against two baselines that were given every reasonable advantage.

We are deliberately narrow. This is an empirical systems study on one corpus with one model and twenty questions. It contributes measurements and diagnosed failure modes, not a new algorithm. Our contributions are:

- **A corpus and benchmark with hard entity identity.** Twelve monthly EDGAR feeds ingested into a relational store and a graph (Table 1), with 20 questions in four categories. The multi-hop subjects were selected on measured evidence: the surname must appear in *zero* risk-factor sections and must be shared by two or more distinct filer identifiers, so that resolving identity by name is demonstrably wrong rather than merely inelegant (Section 5).
- **A matched three-way comparison.** 180 programmatically scored attempts under one model at temperature 0, with the fan-out guardrail and repair policy applied identically to both query-writing arms (Section 4), and with two of the four categories chosen so that the graph should lose.
- **A result that contradicts our expectation, reported with the same prominence as the wins.** Vector RAG lost the single-document category it was expected to win, passing 3 of 15 attempts where GraphRAG passed 15 of 15 (Section 7).
- **A published result of ours that turned out to be our own defect.** We first reported cross-document entity resolution as a loss for the graph. A name-normalisation convention was undocumented in both schema descriptions; documenting it identically and changing nothing else moved three of four categories (Section 9.2, Table 8). We show both measurements, because a repair that moves a headline in the author’s favour should be visible.
- **A cost accounting.** GraphRAG’s accuracy is bought: 520,929 tokens against text-to-SQL’s 144,068, and a repair call triggered on 19 of 60 attempts against 5 of 60 (Table 3, Table 5). We also report the reverse cost: text-to-SQL’s generated joins reached 274s on a single attempt.
- **A safety result that runs against the obvious intuition.** Giving text-to-SQL a better schema description made it more dangerous, not less: its confident falsehoods rose from 2 to 6, because knowing the schema converted refusals into assertive multi-hop answers that conflate people who share a surname. GraphRAG made no false statement in either measurement (Section 7).

Section 2 places the work; Section 3 and Section 4 describe the corpus and the three architectures; Section 5 gives the protocol; Section 6 reports what happened; Section 7 explains the failures; Section 8 states what these measurements cannot support.

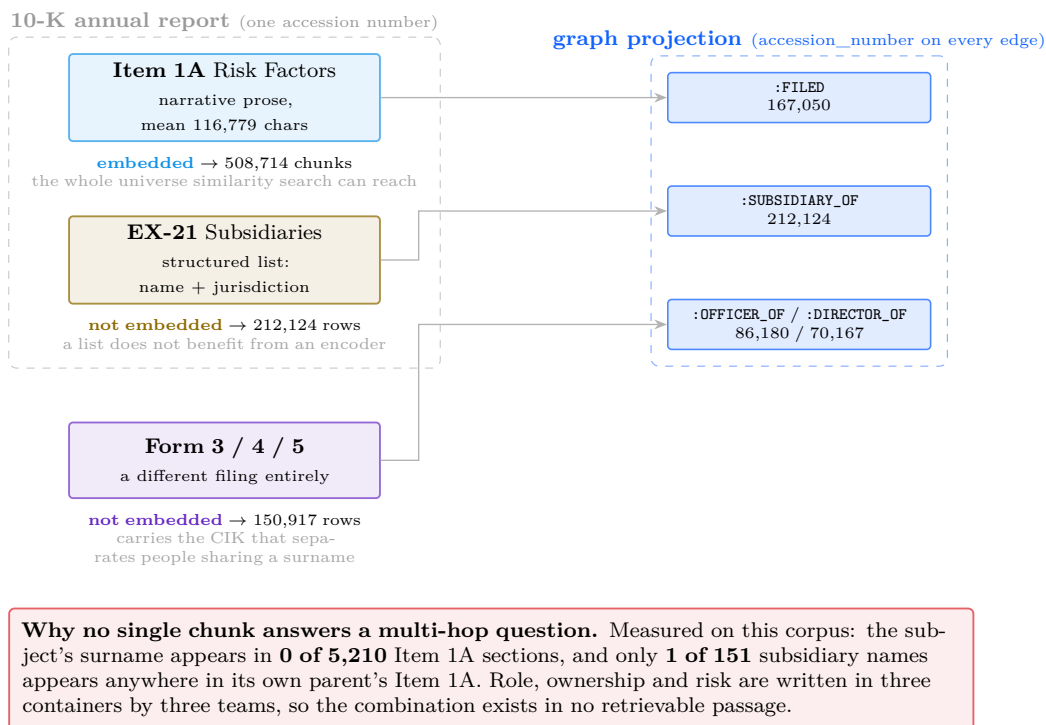


Figure 1: The three facts a multi-hop question needs live in three different containers. Item 1A is narrative prose and is the only section embedded; EX-21 is a structured list attached to the same filing; insider roles are in a separate filing type entirely. Every projected edge carries the accession number of the filing that asserts it.

2 Related Work

Retrieval-augmented generation. Grounding a generator in retrieved passages was formalized by Lewis et al. [2] and rests on dense retrieval [7] and sentence-level encoders [8]. The literature since is large [4], and much of it optimizes the ranking step: better encoders, rerankers, hybrid sparse-dense scoring, query rewriting. Our negative result on similarity search is orthogonal to that line of work. We do not claim the retriever ranked badly; we claim that on questions whose answer spans documents, no ranking of passages can return evidence that no passage contains, and that entity membership is not a property the embedding encodes.

Structured querying as a baseline. Translating questions into executable queries has its own long line, from Zhong et al. [9] to the cross-domain Spider benchmark [10]. We treat text-to-SQL not as a competitor to be defeated but as the strongest available baseline, and we give it the complete schema including the tables that make the multi-hop answers reachable by joins. This matters for the interpretation of our results: text-to-SQL wins one of the four categories outright and beats our graph implementation in a second.

Knowledge graphs and graph-based retrieval. Knowledge graphs [11] predate the current interest, and “GraphRAG” is now an overloaded label [6]. The best-known formulation [5] builds a community structure over an LLM-extracted graph and is aimed at query-focused summarization — global questions such as “what are the themes in this corpus”. Our design targets a different shape. The graph is built from filing metadata and structured exhibits rather than by LLM extraction; the traversal resolves *which entities* are in scope, and the answer text is then fetched by primary key from the relational store rather than summarized from the graph. Only the first hop is LLM-authored; expansion is deterministic, for reasons we quantify in Section 7.

Multi-hop question answering. HotpotQA [12] and MuSiQue [13] established multi-hop evaluation, generally over Wikipedia, where the hops are between topically related articles and entity names are largely unambiguous. Our corpus differs in the property that drives our main result: entity *identity* is genuinely ambiguous. Three distinct people in this corpus share the surname of

our featured subject, and only a stable identifier separates them. A benchmark where names resolve cleanly cannot expose the failure mode we observe.

Evaluation. Automated RAG evaluation increasingly uses an LLM judge [14]. We deliberately do not, because the object under study is whether a system states falsehoods, and hallucination in the judge [15] would be indistinguishable from hallucination in the system. Our scoring is a deterministic function of required entities, forbidden entities, keyword presence and citation correctness (Section 5). This buys reproducibility at the cost of rewarding only the answers we anticipated.

Finally, we position this paper honestly: it is a single-corpus empirical study whose value is in its measurements and its diagnosed failures, not in a new retrieval algorithm.

3 Corpus Construction

3.1 Source and document structure

The corpus is drawn from the U.S. Securities and Exchange Commission’s EDGAR system [1], which publishes every mandatory disclosure filed by every U.S. public company. We ingested twelve monthly XBRL feeds labelled 2025-09 through 2026-08. Filing dates in the resulting corpus span 2025-08 to 2026-08, because a monthly feed includes filings dated at the end of the preceding month; Figure 2b shows the distribution, with the small 2025-08 bucket being that spillover.

The structural point that motivates this paper is that an annual report is not one document. A Form 10-K is a container: a mandated sequence of numbered *Items*, plus attached *Exhibits*, filed under a single accession number. Three containers matter here.

Item 1A, Risk Factors, is narrative prose in which a company enumerates what could go wrong. It is the longest narrative section, averaging 116,779 characters and reaching 1,060,461 in this corpus (Table 2). It is the only section we embed.

Exhibit 21 is an attachment, not a section, required by Item 601(b)(21) of Regulation S-K. It lists subsidiaries and their jurisdiction of incorporation, so it parses into rows rather than prose. Two properties matter. First, the distribution is extremely skewed: 275 filers disclose a single subsidiary while 62 disclose more than 500, and the largest discloses 2,780 (Figure 2a). Second, and more consequentially, *EX-21 is incomplete by design*: rule 601(b)(21)(ii) permits a filer to omit any subsidiary that is not a “significant subsidiary”. The exhibit is therefore a floor on corporate structure and never a census, and the graph inherits that limit. An absent subsidiary is not evidence that none exists.

Forms 3, 4 and 5 are separate filings, not part of the 10-K. Officers, directors and ten-percent owners report their positions and transactions in structured XML. This is the container that makes identity resolvable, because each filing carries the insider’s own Central Index Key (CIK) — a stable SEC identifier. Two directors who share a surname are separable on CIK even though their names are not.

Figure 1 summarizes the arrangement and the measured disjointness that follows from it.

3.2 Ingestion

The pipeline runs in nine stages: fetch and parse the monthly feeds; fetch the documents each filing manifests; extract narrative Items from HTML; parse EX-21 and the ownership forms; stage to object storage; load the relational store; embed; and build indexes and the graph projection. Figure 3 gives the wall-clock breakdown, totalling roughly 252 minutes. Two stages dominate: document fetch (118 minutes) and embedding (87 minutes), together about 82% of the total. The fetch cost is externally bounded — EDGAR’s fair-access policy caps automated requests at ten per second per requester [1] — so no amount of parallelism removes it.

The filings manifest 1,904,128 documents in total. We fetched and parsed the 10,592 that carry the narrative and exhibit content the benchmark needs, amounting to 19.7 GB of raw HTML, which

reduces to a 3,803 MiB relational store plus a 1.0 GB vector index. Table 1 gives the resulting counts.

3.3 Schema

The relational store is PostgreSQL. Filings and companies are keyed by accession number and CIK. `filing_section` holds one row per extracted Item with its text; `section_chunk` holds the embedded chunks; `subsidiary` holds parsed EX-21 rows; `reporting_owner` holds parsed ownership rows; and `filing_auditor` holds auditor attestations.

The graph is a projection of the same data, not a second source of truth. It holds 430,257 nodes across five labels — `Filing` (167,050), `Subsidiary` (200,869), `Person` (51,558), `Company` (10,511) and `AuditFirm` (269) — and 561,144 edges across seven types: `:SUBSIDIARY_OF` (212,124), `:FILED` (167,050), `:OFFICER_OF` (86,180), `:DIRECTOR_OF` (70,167), `:OWNS_SHARES` (19,542), `:AUDITED_BY` (5,835) and `:RESOLVES_TO` (246).

Two design decisions in that schema carry most of the paper’s weight.

Every edge carries the accession number of the filing that asserts it. Provenance is a property of the relationship rather than of a document, which is what lets an answer cite the exact filing supporting each individual claim rather than the set of documents that were retrieved.

:RESOLVES_TO is the entity-resolution layer, and it is thin. It links a subsidiary named in an EX-21 to a company that files in its own right, a fact no single document states. There are 246 such edges against 200,869 subsidiary nodes — a coverage of 0.12% — because EX-21 carries no CIK for a subsidiary, so the link must be established by name matching. We show in Section 7 that this thinness is a direct cause of the graph losing the one category built to exercise it.

Entity	Count
Filings (all form types)	167,050
Distinct filers (companies)	10,511
Documents in filing manifests	1,904,128
Documents fetched and parsed	10,592
10-K narrative sections extracted	23,649
Item 1A chunks embedded	508,714
EX-21 subsidiary rows	212,124
Form 3/4/5 ownership rows	150,917
Distinct insiders	51,558
Auditor attestations	5,835
Graph nodes	430,257
Graph edges	561,144
Relational store on disk	3,803 MiB
HNSW index	1.0 GB

Table 1: The loaded corpus. Twelve monthly EDGAR feeds, 2025-09 through 2026-08. Node and edge counts are live counts from the graph; the per-type edge breakdown in Section 3 sums exactly to the total shown here.

4 System Architecture

Three retrieval arms answer the same question over the same stores. Figure 4 shows all three.

4.1 Text-to-SQL

The model is shown the complete relational schema, including the `subsidiary` and `reporting_owner` tables, and writes a SQL query; the query is executed; the resulting rows are given back to the model,

Item	Section	Filings	Total chars	Mean	Max
1	Business	5,539	352,550,560	63,649	717,422
1A	Risk Factors	5,210	608,417,255	116,779	1,060,461
1B	Unresolved Staff Comments	63	2,000,524	31,754	127,289
2	Properties	2,683	12,281,341	4,577	390,319
3	Legal Proceedings	1,518	5,126,849	3,377	117,185
7	Management’s Discussion & Analysis	5,609	294,551,458	52,514	473,035
7A	Market Risk	3,027	16,460,821	5,438	276,843

Table 2: Narrative sections extracted per 10-K Item. Only Item 1A (bold) is embedded, giving 508,714 chunks. Its mean length exceeds the encoder’s 256-word-piece window by roughly two orders of magnitude, which is why chunking is forced rather than optional.

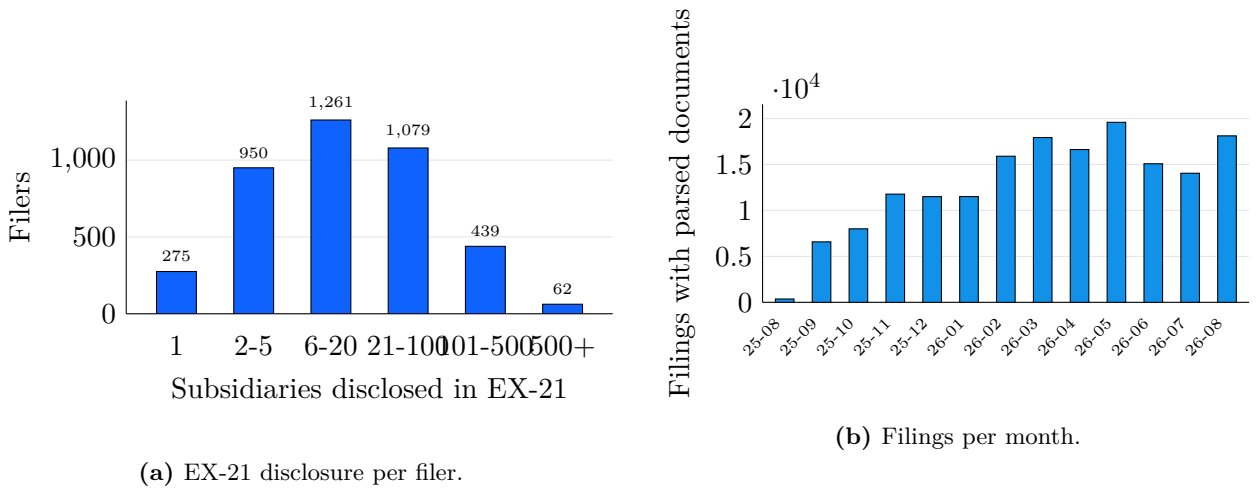


Figure 2: Corpus shape. (a) EX-21 disclosure is extremely skewed — 275 filers list one subsidiary, 62 list more than 500 — so any generated LIMIT over a join to this table starves whole entities unless rows are balanced per entity first. (b) The small 2025-08 bucket is spillover, since a monthly feed contains filings dated at the end of the preceding month.

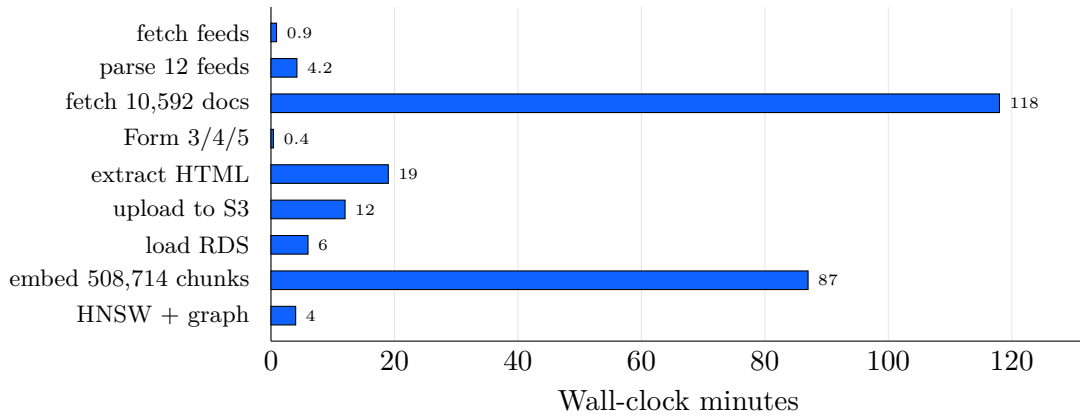


Figure 3: Ingestion wall-clock, about 252 minutes total. Document fetch and embedding are roughly 82% of it. The fetch component is bounded externally by EDGAR’s ten-requests-per-second fair-access policy and cannot be parallelized away.

which writes the answer. Two LLM calls.

This is deliberately a strong baseline rather than a straw man. Because the schema is complete, every multi-hop answer in the benchmark *is* reachable by joins. What the model must do unaided is choose the join path, decode a multi-valued relationship string, and recognize that traversing an ownership chain of unknown depth requires a recursive CTE. The schema documentation also states the surname-first convention used for person names — an asymmetry in an earlier iteration, where only the graph documentation mentioned it, is recorded among the harness defects in Section 9.

4.2 Vector RAG

The question is embedded and the $k=8$ nearest chunks are retrieved by cosine similarity from an HNSW index [3, 16]; the chunks are given to the model, which answers. One LLM call, and no query language, therefore no repair path.

The encoder is `all-MiniLM-L6-v2`, a six-layer 22.7M-parameter model distilled in the MiniLM family [17] and trained as a sentence encoder [8]. It emits 384 dimensions with mean pooling and L^2 normalization, so cosine similarity reduces to a dot product.

One configuration detail is load-bearing. The encoder accepts 256 word-pieces, roughly a thousand characters, while Item 1A averages 116,779 characters. The section therefore cannot be embedded whole: text is chunked at 1,400 characters with 200 characters of overlap, so a single risk-factor section becomes on the order of a hundred vectors, and 5,210 sections become 508,714 chunks. We embed only Item 1A. That is not a handicap imposed on the baseline but the opposite — it is the only section any benchmark question asks about, so restricting the index concentrates the candidate pool and makes the vector arm *stronger* than it would be with all seven Items embedded.

4.3 GraphRAG

Two LLM calls, as with text-to-SQL, but the work between them is split differently.

1. **Identify.** The model writes a Cypher query whose job is to determine *which entities* the question concerns — typically resolving a person to a CIK and finding the companies where they hold a role, with their relevant filings.
2. **Expand (deterministic).** The system, not the model, expands each identified entity: for every company returned, fetch its subsidiaries and its risk-factor filings.
3. **Fetch text by key.** The accession numbers from the traversal are used to select section text by primary key, and windows of ± 900 characters around matched terms are extracted, at most three per filing. This is a lookup, not a search: the graph has already decided which documents are in scope, so there is nothing to rank and nothing to miss.
4. **Answer.** The model receives the graph facts and the exact passages, and writes the answer with citations.

Stage 2 being system-authored rather than LLM-authored is the design’s central claim, and it is a response to a measured failure rather than an aesthetic preference. An LLM writing a single query over a fan-out join will attach a `LIMIT`, and one entity’s hundreds of child rows then consume the entire budget — after which the model reports, fluently and wrongly, that the other entities have no subsidiaries or do not exist. Splitting identification from expansion makes that class of error structurally impossible rather than something to be prompted against.

4.4 Shared guardrails

The same failure threatens both query languages, so the same two transforms are applied to both, identically. Any trailing `LIMIT` in a generated query is raised to 2,000 rows before execution; the

result set is then balanced to at most 6 rows per entity and 48 rows overall, where the entity column is detected by name. Applying these to SQL and Cypher alike means the comparison measures retrieval architecture rather than which prompt happened to dodge the trap.

Both query-writing arms also get exactly one repair attempt, triggered when a query raises an exception or returns zero rows: the model sees the error text and rewrites once. There is no second repair. Section 6 shows the two arms used this allowance very unequally.

4.5 Model access

All three arms call a single OpenAI-compatible endpoint provided by Bifrost, an open-source LLM gateway, which routes to DeepSeek or to AWS Bedrock. The benchmarked configuration is `deepseek-v4-flash` at temperature 0 with reasoning disabled, 700 output tokens for query generation and 1,500 for answers.

Routing through one endpoint is a methodological device, not a convenience. It makes “all three arms used the same model with the same settings” a structural property of the deployment rather than a convention maintained by discipline: the model is a request parameter read from one place by all three arms. It also makes the cross-model probe in Section 7 a configuration change rather than a code change. Reasoning was disabled because an uncapped reasoning trace consumed roughly 33,000 completion tokens per call, which `max_tokens` does not bound; the setting is identical for every arm, so the comparison is unaffected.

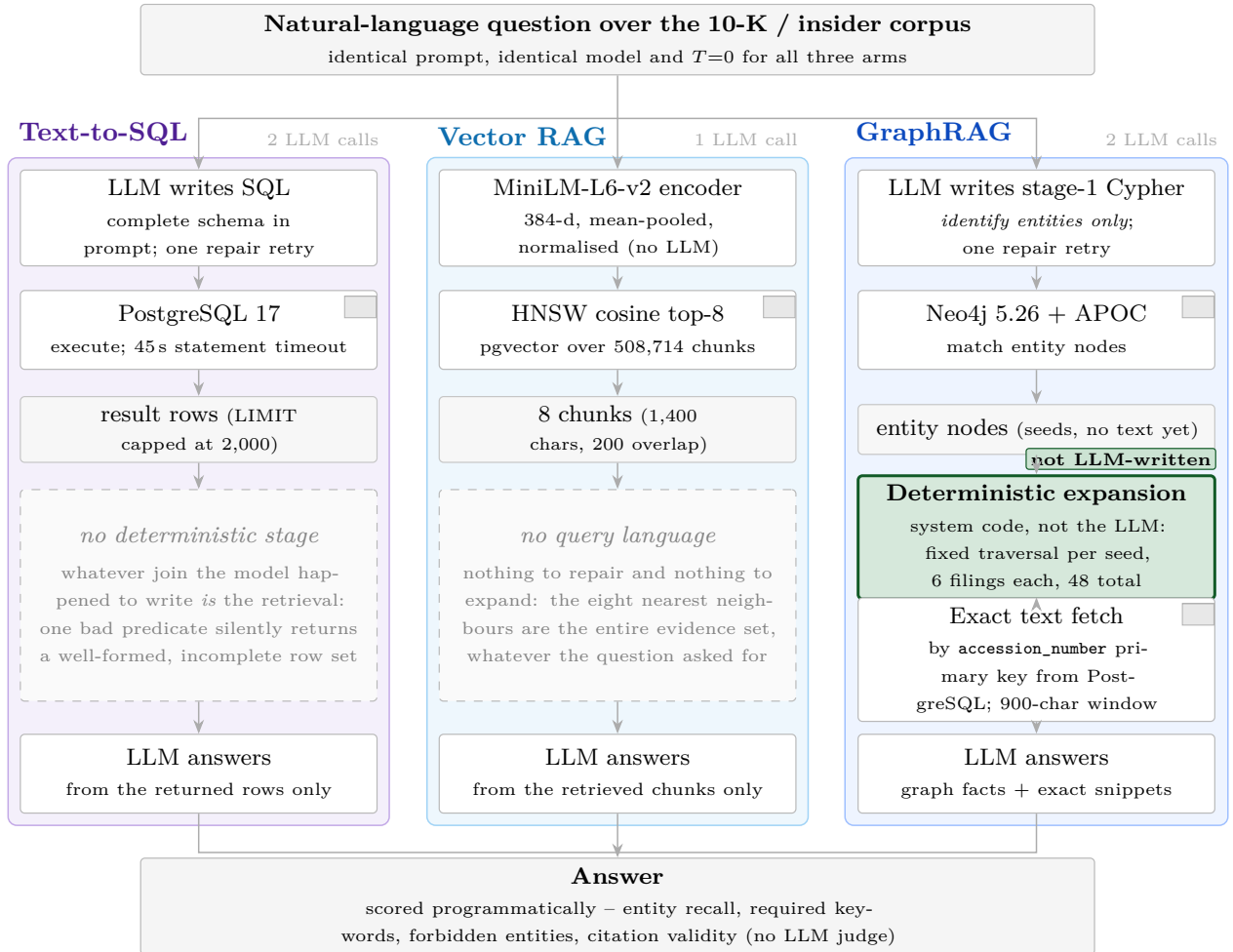


Figure 4: The three retrieval arms over shared stores. Text-to-SQL and GraphRAG use two LLM calls each; vector RAG uses one and has no query language to repair. In GraphRAG only the first hop is LLM-authored: entity expansion is deterministic and system-authored, which is what prevents one entity’s fan-out from consuming the row budget and the model then reporting that the other entities do not exist.

5 Experimental Design

5.1 Four question shapes

The benchmark has 20 questions in four categories of five, listed in full in Appendix A. Each category carries a pre-registered expectation of which architecture should win, and two of the four were chosen so that the graph is the *wrong* tool:

- **T1, single-document semantic.** What did one named company disclose about one topic in its risk factors? The whole answer lies inside one document. Expected winner: vector RAG or GraphRAG.
- **T2, structured aggregation.** Which audit firm serves the most companies; which jurisdiction of incorporation is most common. These are `GROUP BY` queries over typed columns. Expected winner: text-to-SQL, and a traversal should add nothing.
- **T3, multi-hop relational plus text.** For every company where a named person holds an insider role, list its subsidiaries and summarize its reported cybersecurity risks. Expected winner: GraphRAG.
- **T4, cross-document entity resolution.** Is a named subsidiary also an SEC filer in its own right, and which parent lists it? Expected winner: GraphRAG.

5.2 Selecting subjects that make identity hard

The T3 subjects were not chosen for convenience. Two conditions had to hold simultaneously, both checked against the full-text index over all 5,210 risk-factor sections:

1. the surname appears in *zero* risk-factor sections, so no similarity search can reach the person at all; and
2. two or more distinct filer identifiers share that surname, so resolving identity by name is demonstrably wrong rather than merely imprecise.

Candidates were rejected on the first condition when the surname was too common in the text (32, 49, 63 and 109 matching sections respectively) and on the second when the surname turned out to be unique in the corpus. One unique-surname subject was retained deliberately as a control: same question shape, no identity trap.

5.3 Scoring

Scoring is a deterministic function of the answer text. There is no LLM judge, for the reason given in Section 2. An attempt **passes** when all four conditions hold:

1. every required entity is named (entity recall = 1.0);
2. at least one required topical keyword is present;
3. no forbidden entity is named — these are the specific wrong companies that a name-based rather than identifier-based resolution would return;
4. where ground-truth citations exist, at least one is correct.

The fourth condition is what makes provenance count. Without it, a model that named the right companies but cited nothing would score identically to one that cited the exact filings. Failures are then labelled: **falsehood** when the answer is not correct, is not a refusal, and either names a forbidden entity or cites a filing outside the valid set; **uncited** when the required entities and keywords are present with nothing false but no correct citation; **refused** when a regex over the

answer detects an explicit statement that the question cannot be answered from the evidence; and **fail** otherwise. The labels partition the attempts, so the columns of Table 3 sum to 60.

Aggregate questions have no ground-truth citation set. For those, any accession the model cites is legitimate provenance and is not counted against it — an earlier version of the scorer did count it, and Section 9 records what that cost.

5.4 Three attempts, all of them counted

Every question was asked three times of every architecture, giving $20 \times 3 \times 3 = 180$ scored attempts. The three attempts are *independent repeats to measure consistency, not retries*: every attempt is scored and every attempt counts, and there is no best-of-three anywhere in the reported numbers.

This is not ceremony. Even at temperature 0 the generations are not deterministic, and **9 of the 60** question-by-architecture cells returned different labels across their three attempts — 7 for text-to-SQL, 2 for GraphRAG and 0 for vector RAG. Text-to-SQL was inconsistent on four of the five multi-hop questions, which is precisely where its false statements occur: an evaluation drawing a single attempt per question could record an honest refusal on the same question where another draw asserts something false.

We therefore report two metrics. **Attempts passed** out of 60 is the raw rate. **Questions solved** out of 20 credits a question only when all three of its attempts passed, and is the stricter reading: an answer that is correct once in three is not a capability a practitioner can deploy. The two are not related by a fixed ratio — an architecture with q questions solved has at least $3q$ attempts passed, and the excess is contributed by questions solved inconsistently.

5.5 Fairness controls

A rigged baseline would prove nothing, so we record exactly what was equalized: identical model, temperature, reasoning setting and answer-token budget for all three arms; an equal LLM call budget of two calls for the query-writing arms and one for vector RAG, which has no query to write; the complete schema shown to text-to-SQL so that every multi-hop answer is reachable by joins; the surname-first naming convention documented in both schema descriptions; one repair attempt for each query-writing arm; and the identical fan-out guardrail on both query languages. Two of the four categories were selected so that the graph should lose them.

We note one asymmetry we did not eliminate, because it favours the graph and we would rather report it than hide it. The repair trigger fires on an exception or zero rows. An over-broad SQL join returns *many* rows and therefore never trips it, even when the answer is wrong, whereas an exact graph property match that misses returns nothing and earns a free second attempt. The same rule confers unequal benefit; Table 5 quantifies the result.

6 Results

Table 3 gives the headline numbers and Table 4 the per-category breakdown. Figure 5a plots the latter and Table 6 shows every one of the 180 attempts individually, unaggregated.

Summed over the question set, GraphRAG solved 16 of 20 questions on all three attempts and passed 50 of 60 attempts; text-to-SQL solved 10 and passed 36; vector RAG solved 3 and passed 9. These totals weight four hand-chosen categories equally, and GraphRAG lost one of them outright, so the aggregate is a property of this question mix rather than an overall ranking of the architectures. The per-category results are the informative ones.

6.1 By category

T1, single-document semantic. GraphRAG passed 15 of 15. Vector RAG, the expected winner, passed 3 of 15 — all from one question — with the other 12 attempts naming the right company and the right risks but citing no filing. Text-to-SQL passed 7 of 15, using full-text ranking to locate the passage; its remaining attempts were 6 uncited and 2 refusals. Note that both baselines here fail predominantly by omitting provenance rather than by being wrong, which is a materially different failure from asserting something false.

T2, structured aggregation. Text-to-SQL passed 15 of 15, as predicted. GraphRAG passed 6 of 15, solving 2 of 5 questions, with 8 attempts returning wrong answers and 1 refusing. Vector RAG passed 0 of 15: an aggregate over 10,511 companies is not present in any eight retrieved chunks, and it said so rather than guessing. This is the one category the graph loses, and it loses it badly and expensively.

T3, multi-hop relational plus text. GraphRAG passed 14 of 15; both baselines passed 0 of 15. This is the category the design targets. It is also where every one of the benchmark’s confident falsehoods occurred, all six of them text-to-SQL.

T4, cross-document entity resolution. GraphRAG passed 15 of 15 and text-to-SQL 14 of 15; vector RAG passed 6 of 15. An earlier version of this measurement recorded GraphRAG at 9 of 15 here and reported the category as a loss for structural retrieval. That was a defect in our schema documentation rather than a property of the graph; Section 9.2 and Table 8 describe the repair and its effect.

6.2 Outcome composition

Figure 5b decomposes all 60 attempts per arm, and the composition matters as much as the pass rate because the failure modes are not interchangeable.

Vector RAG refused 39 times and was uncited 12 times. That is a low score but a benign profile: asked about a company whose chunks it could not find, it either abstained or answered without provenance, and it never asserted anything false. Text-to-SQL was uncited 10 times and made **6 false statements**. GraphRAG was never uncited and refused only once, which follows from its construction: the traversal returns accession numbers before any text is read, so every claim arrives with a key attached.

6.3 Cost and latency

Figure 6b plots tokens against attempts passed. GraphRAG consumed 520,929 tokens to text-to-SQL’s 144,068 and vector RAG’s 118,185, giving it the highest cost per passing attempt of the three (Table 3). Its accuracy is bought, not free. The extreme case is T2, where GraphRAG spent 12.2 times text-to-SQL’s tokens to score 6 of 15 against 15 of 15 — on the category it was expected to lose.

The reverse cost is easy to overlook. Figure 6a plots mean latency on a log scale. Text-to-SQL averaged 10.0s against GraphRAG’s 4.6s and vector RAG’s 1.3s, but the overall mean hides the shape: its worst single attempt took 274s, because the generated queries join **subsidiary**, **filing_section** and **reporting_owner** without narrowing first. Unbounded LLM-authored SQL is a cost risk against one’s own warehouse independently of whether the answer is correct, which is why the live service runs with a statement timeout that this measurement deliberately disabled (Section 9.2).

6.4 Repair usage

Table 5 and Figure 7 report how often each arm used its single repair allowance. Vector RAG has no query to repair and used none. As noted in Section 5, the zero-row trigger is not neutral between the two query languages: an over-broad SQL join returns many rows and never trips it, while an exact graph property match that misses returns nothing and earns a free second attempt. The direction of that bias favours the graph, and we report the usage rather than claim it away.

	Text-to-SQL	Vector RAG	GraphRAG
Questions solved on all 3 attempts	10/20	3/20	16/20
Questions solved inconsistently	3	0	1
Attempts passed (of 60)	36	9	50
Confident falsehoods	6	0	0
Correct but uncited	10	12	0
Explicit refusals	8	39	1
Mean evidence supplied (chars)	1,795	8,557	6,084
Mean latency (s)	10.0	1.3	4.6
Max latency (s)	274.5	4.6	8.6
Total tokens	144,068	118,185	520,929
Tokens per passing attempt	4,001	13,131	10,418
LLM calls	125	60	139

Table 3: Aggregate results over 180 scored attempts. *Questions solved* credits a question only when all three of its attempts passed. Bold marks the best value in a row. The outcome labels partition the attempts, so pass, falsehood, uncited, refused and the residual failures sum to 60 per arm.

Type	Question class	Text-to-SQL		Vector RAG		GraphRAG	
		Q	A	Q	A	Q	A
T1	single-document semantic	1/5	7/15	1/5	3/15	5/5	15/15
T2	structured aggregation	5/5	15/15	0/5	0/15	2/5	6/15
T3	multi-hop relational + text	0/5	0/15	0/5	0/15	4/5	14/15
T4	cross-document entity resolution	4/5	14/15	2/5	6/15	5/5	15/15

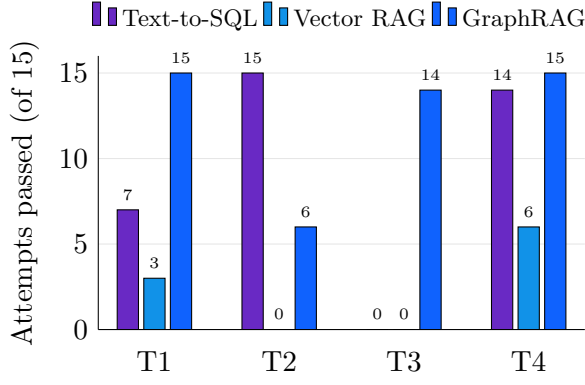
Table 4: Per-category results. **Q** = questions solved on all three attempts (of 5); **A** = attempts passed (of 15). Two of the four categories were pre-registered as ones the graph should lose. It lost T2 as expected and T4 against expectation.

Architecture	Overall	T1	T2	T3	T4
Text-to-SQL	5/60	3/15	1/15	0/15	1/15
Vector RAG	0/60	0/15	0/15	0/15	0/15
GraphRAG	19/60	1/15	9/15	0/15	9/15

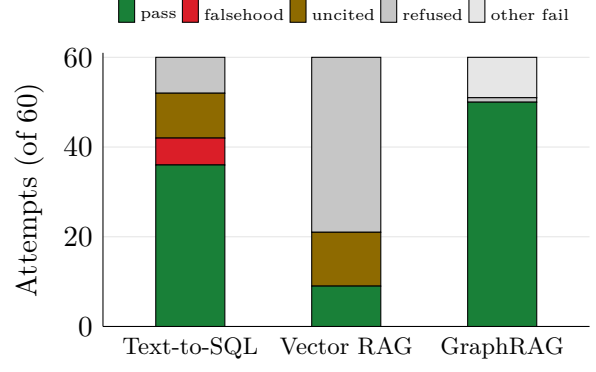
Table 5: Use of the single repair allowance, which both query-writing arms receive on identical terms. GraphRAG leaned on it nearly five times as often, and on T4 needed it on every attempt. Vector RAG has no query language to repair.

7 Failure Analysis

The failures are the most informative part of this study, and two of them contradict what we expected before running it.

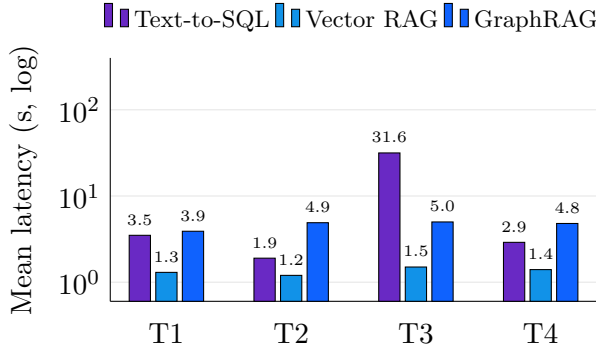


(a) Attempts passed per category.

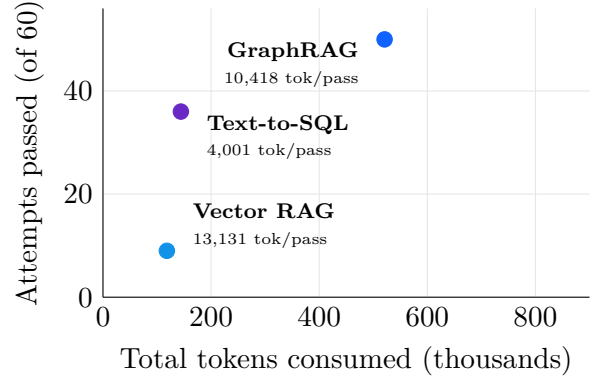


(b) How all 60 attempts resolved.

Figure 5: Results. The composition in (b) matters as much as the rate in (a): vector RAG’s 45 refusals are a benign failure, text-to-SQL’s 16 uncited answers are right but unverifiable, and GraphRAG was never uncited because the traversal returns accession numbers before any text is read.



(a) Mean latency per category (log).



(b) Tokens against attempts passed.

Figure 6: The economics. (a) Text-to-SQL’s T3 mean is the SQL and not the model: the generated queries join three tables without narrowing first. (b) GraphRAG’s accuracy is bought: it has the highest token cost per passing attempt of the three, the exact figures being in Table 3.

Q	Text-to-SQL	Vector RAG	GraphRAG
T1-1	pass, pass, correct but uncited	correct but uncited, correct but uncited, correct but uncited	pass, pass, pass
T1-2	pass, pass, pass	pass, pass, pass	pass, pass, pass
T1-3	correct but uncited, correct but uncited, correct but uncited	pass, pass, pass	pass, pass, pass
T1-4	pass, refused, pass	correct but uncited, correct but uncited, correct but uncited	pass, pass, pass
T1-5	correct but uncited, correct but uncited, correct but uncited	correct but uncited, correct but uncited, correct but uncited	pass, pass, pass
T2-1	pass, pass, pass	refused, refused, refused	other failure, other failure, other failure
T2-2	pass, pass, pass	refused, refused, refused	pass, pass, pass
T2-3	pass, pass, pass	refused, refused, refused	other failure, other failure, other failure
T2-4	pass, pass, pass	refused, refused, refused	other failure, pass, other failure
T2-5	pass, pass, pass	refused, refused, refused	pass, pass, pass
T3-1	pass, falsehood, pass	refused, refused, refused	pass, pass, pass
T3-2 *	falsehood, falsehood, pass	refused, refused, refused	pass, pass, pass
T3-3	pass, refused, pass	refused, refused, refused	pass, pass, pass
T3-4	correct but uncited, correct but uncited, correct but uncited	refused, refused, refused	pass, pass, pass
T3-5	correct but uncited, correct but uncited, correct but uncited	refused, refused, refused	pass, pass, other failure
T4-1	pass, pass, pass	refused, refused, refused	pass, pass, pass
T4-2	pass, pass, pass	refused, refused, refused	pass, pass, pass
T4-3	pass, pass, pass	refused, refused, refused	pass, pass, pass
T4-4	pass, pass, pass	pass, pass, pass	pass, pass, pass
T4-5	pass, pass, pass	pass, pass, pass	pass, pass, pass

Table 6: Every one of the 180 attempts, unaggregated; three glyphs per cell are the three attempts. ■ pass, ■ falsehood, ■ correct but uncited, □ refused, × other failure. ★ marks the featured multi-hop question. 10 of these 60 cells did not return the same label on all three attempts.

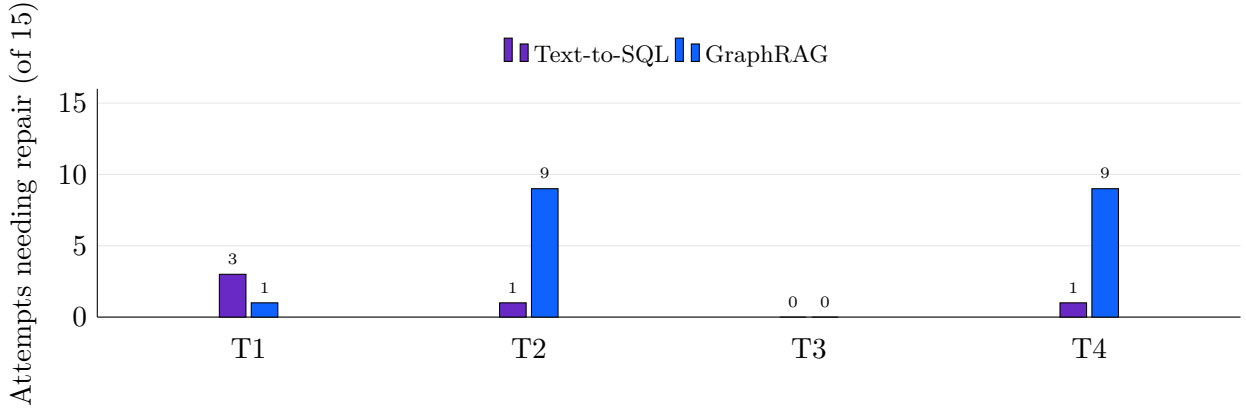


Figure 7: Repair frequency by category. GraphRAG required a repair on all fifteen T4 attempts, which is the entity-resolution category it lost.

7.1 Vector RAG lost the category it was expected to win

T1 asks what one named company disclosed about one topic in its risk factors. The answer is entirely inside a single document, retrieval is a ranking problem, and similarity search should be at its best. It passed 3 of 15 attempts, all from one question.

The mechanism is visible in the retrieved set. Asked what manufacturing and supply-chain risks *Moderna* disclosed, the top-8 chunks came from Entegris, Hayward Holdings, Open Text, Arrowhead Pharmaceuticals and NVIDIA. Every one of those chunks genuinely discusses supply-chain risk. The retriever did what it was asked. The problem is that “belongs to Moderna” is not a property the embedding encodes, so the company name in the question cannot outweigh topical similarity across 508,714 chunks drawn from 5,210 filers.

The decisive detail is that **this question passed on a single month of filings**. The failure appeared when the corpus grew. It is therefore a scale-induced limitation of entity filtering by similarity, not a tuning error, and it is the strongest argument in this paper for structural retrieval — and one we did not anticipate.

We are careful about the boundary of this claim. This experiment cannot fully separate “embeddings cannot filter by entity” from “this retriever configuration cannot”, because the 1,400-character chunks exceed the encoder’s 256-token window and k was fixed at 8. A hybrid retriever with a metadata filter on CIK would very likely fix T1 — which is precisely the point, since a metadata filter on a stable identifier *is* structural retrieval, arriving by another route.

7.2 The category we got wrong, and what it cost

We first published T4 — cross-document entity resolution — as a loss for structural retrieval: text-to-SQL 14 of 15 attempts against GraphRAG’s 9, in the category written for the graph. It was not a loss. It was a defect in our schema documentation, described in Section 9.2. Both stores normalise names by stripping corporate suffixes, neither schema description said so, and only the graph arm was exposed because it matched the property exactly while text-to-SQL matched the raw column with `ILIKE`. Repaired, GraphRAG passes 15 of 15.

We keep the episode in the paper for three reasons, none of them flattering.

First, it shows how narrow the margin is between an architectural finding and a documentation bug. Nothing about the traversal changed. One sentence of schema prose was worth six attempts and a category.

Second, the repair was not free elsewhere and not confined to the graph. The same sentence added to the SQL schema description moved text-to-SQL’s single-document score from 0 of 15 to 7 of 15, so our original report understated *both* the graph and the strongest baseline. Table 8 gives

the full movement. A fix that only helped the author’s preferred system would deserve suspicion; this one helped the baseline more in raw attempts.

Third, and most usefully, it made the baseline *more dangerous*. Text-to-SQL’s confident falsehoods rose from 2 to 6. Better schema knowledge did not make it more careful; it converted refusals into assertive multi-hop answers, and on this corpus an assertive multi-hop answer keyed on a person’s name is how you conflate three different people called Lozano. The safety property did not improve with capability. It degraded with it.

7.3 The remaining T4 failure

One T4 question still fails for the graph on some attempts, and we did not chase it. *Alexander's, Inc.* normalises to `alexander s` because the apostrophe is replaced rather than removed, so a match on the normalised form requires reproducing that specific transformation. We fixed the documented convention once, symmetrically, and re-measured once. Continuing to tune until a particular question passes is how a benchmark stops measuring anything, so the residual failure is reported rather than removed.

7.4 GraphRAG lost aggregation, as predicted

Text-to-SQL scored 15 of 15 on T2 and GraphRAG 6 of 15. Unlike the previous two, this outcome was expected — T2 exists in the benchmark precisely because a traversal should add nothing to a `GROUP BY` — but the manner of the loss is instructive.

Asked for the most common jurisdiction of incorporation among all disclosed subsidiaries, the generated Cypher grouped by company rather than by jurisdiction, returned per-company counts, and the model reported Delaware with 2,415 subsidiaries. Delaware is the right answer; 2,415 is one company’s count rather than the 62,504 across the corpus. Right entity, wrong aggregate. The answer also attached 190 accession numbers to a question that needs none.

For counting over typed columns the graph adds no information and the extra hop adds a place to go wrong, at 24 times the token cost.

7.5 Every falsehood was a surname collision

Six of the 180 attempts stated something verifiably false. All six were text-to-SQL, all six on the multi-hop category, and all six are the same error, spread across four of its five questions (T3-1, T3-2, T3-3, T3-4). Asked about *Monica Lozano*, the model reported companies including Mondelez and Ternium; those belong to Mariano and Santiago Lozano, different people who share a surname. Asked about *William Giles*, it reported ten companies.

The SQL was not malformed and the model was fluent and confident. It matched on name because name is what the question supplied and what the text contains. This is the exact failure that resolving identity to an identifier before reading any text prevents structurally rather than probabilistically: GraphRAG passed 14 of 15 attempts on this category and made no false statement in either measurement, because the other Lozanos never entered its result set.

Two observations sharpen this. The first is that the count *rose* when we improved the baseline’s schema documentation, from 2 to 6; capability and safety moved in opposite directions. The second is a caution about single-attempt evaluation: of the three attempts text-to-SQL made on the featured question, the outcomes were not identical, and an evaluation that drew only one of them could easily have recorded an honest refusal and missed the failure entirely.

7.6 The verdict is model-sensitive

Because all arms route through one gateway (Section 4), we probed the same GraphRAG pipeline on other models. Table 7 reports the featured multi-hop question: the benchmarked model passes, Claude Haiku 4.5 fails because its stage-1 Cypher collapsed to a single company and missed two others, and Amazon Nova 2 Lite refuses.

This is a probe and not a measurement — one question, one attempt each, no re-scoring of the full set — and we draw only the negative conclusion it supports: a single-model benchmark bounds what can be claimed. The architecture constrains what evidence reaches the model, but the model still has to write a correct traversal, and not all of them do.

Model	Verdict	Observed behaviour
DeepSeek V4 Flash (benchmarked)	pass	correct three-company traversal
Claude Haiku 4.5 (Bedrock)	fail	stage-1 Cypher returned TARGET CORP alone; Apple and Bank of America missing
Amazon Nova 2 Lite (Bedrock)	refused	produced no usable traversal

Table 7: Cross-model probe on the featured multi-hop question with the GraphRAG pipeline held constant. One question and one attempt per model, so this is a probe rather than a measurement; it supports only the negative conclusion that a single-model benchmark bounds what can be claimed.

8 Threats to Validity

One corpus, one domain. Every measurement comes from a single snapshot of one filing system. SEC disclosure is unusually favourable to structural retrieval: it is mandated, schematized, and carries a stable public entity identifier in the CIK. Corpora without an authoritative identifier — internal wikis, support tickets, email — would require the entity-resolution step this paper largely assumes, and Section 7 shows that step is our weakest layer. Nothing here should be read as generalizing to such corpora.

Twenty questions is a small sample. Each category rests on five questions, so a category-level rate has a wide confidence interval: a single question changes a category by 20 percentage points at question level and 3 attempts out of 15 at attempt level. The category results are directional, not conclusive. The one result we would defend as strong is T3, where the margin is 15 of 15 against 0 of 15 for both baselines and the mechanism is independently diagnosed.

The questions were written by the author of the graph. This is the most serious threat and it cannot be fully mitigated. We took three measures: two of the four categories were pre-registered as ones the graph should lose, and it did lose both; the T3 subjects were selected by a measured criterion rather than by inspection of results (Section 5); and the ground truth was verified against the loaded tables rather than against model outputs. None of this eliminates the risk that the question set favours the design in ways we cannot see. An independent question set is the obvious next step.

One model, and the verdict moves with it. All reported numbers come from `deepseek-v4-flash`. The probe in Table 7 shows the featured question passing on one model, failing on another and being refused by a third with the retrieval architecture held constant. Our results characterize *architecture* $\times$ *model*, and we did not vary the model across the full 180 attempts.

The graph inherits EX-21 incompleteness. Rule 601(b)(21)(ii) allows filers to omit insignificant subsidiaries, so `:SUBSIDIARY_OF` is a floor on corporate structure. Any question whose correct answer depends on a subsidiary the filer chose not to disclose is unanswerable from this corpus by any of the three arms, and our ground truth is defined against what was disclosed rather than against reality.

Category results move with implementation details. GraphRAG’s originally published loss on T4 turned out to be a documentation defect in our own harness rather than a property of

the graph, and repairing it changed the category (Section 9.2, Table 8). We report the repaired numbers, but the episode is itself a threat: if one such defect was worth a category, others may be latent in results we have not thought to re-examine, and the ones we would be least likely to notice are those that flatter the design. The same logic applies in reverse to the wins — some part of GraphRAG’s T1 and T3 margins may reflect implementation effort we spent on the graph and not on the baselines. `:RESOLVES_T0` coverage of 0.12% remains unrepaired and still bounds what any entity-resolution question can measure here.

Repair asymmetry favours the graph. As recorded in Section 5, the zero-row repair trigger fires for a graph query that misses but not for an over-broad SQL join that returns many wrong rows. GraphRAG used its repair on 34 of 60 attempts against text-to-SQL’s 7, so it received materially more effective assistance from an identical rule.

A single year, and a moving corpus. The window spans twelve monthly feeds. Annual filings cluster seasonally (Figure 2b), so the mix of form types is not stationary within it, and the absolute counts are not reproducible against a later snapshot because EDGAR accumulates.

Undetected ground-truth errors are likely. We found eight defects in our own harness (Section 9), one of them a ground-truth error discovered *after* the results were first computed: a question accepted one of three filers that genuinely list a subsidiary, so all three arms were naming a real parent and being marked wrong. Correcting it and re-scoring the stored answers moved text-to-SQL from 28 to 29, vector RAG from 6 to 9 and GraphRAG from 44 to 45. That we found one after believing the results were final is direct evidence that others may remain, and the correction happened to help the baselines more than the graph.

9 Reproducibility and Engineering Cost

9.1 Software and infrastructure

Both relational and vector retrieval run on PostgreSQL 17 with pgvector 0.8.2 [16], using an HNSW index over `vector_cosine_ops` [3] alongside `pg_trgm` and a GIN `tsvector` index. `hnsw.iterative_scan` is set to `relaxed_order` for reasons given below. The graph is Neo4j 5.26 Community with APOC [18]. Embeddings come from `all-MiniLM-L6-v2` at 384 dimensions with a 256 word-piece limit, chunked at 1,400 characters with 200 of overlap, batch size 256. Generated SQL and Cypher execute under a 45-second statement timeout.

The deployment is a single application host — an AWS `m7g.xlarge`, 4 vCPU and 16 GB, running the graph, the API and the gateway — with PostgreSQL on a `db.t4g.medium` instance and CloudFront terminating TLS. There is no NAT gateway; object-store access goes through a VPC endpoint. Table 9 gives the monthly cost at roughly \$165. The embedding pass was run on a temporarily resized 16-vCPU host at 97 chunks/s and the instance was then scaled back.

Ingestion takes about 252 minutes end to end (Figure 3), of which document fetch and embedding are about 82%. The fetch component is bounded externally by EDGAR’s ten-requests-per-second fair-access policy [1] and cannot be parallelized away.

9.2 Auditing the harness

Building the measurement code surfaced defects that, unnoticed, would each have produced a publishable-looking but incorrect result: a pgvector post-filter that returned 0 of 8 rows while 164 matched, so the vector arm was briefly scored on an empty retriever; an answer-token cap that truncated answers mid-citation, silently converting passes into uncited; a global token counter that attributed every thread’s cost to whichever arm read it first. These were found and fixed before the reported numbers were generated, so they are development history rather than caveats on the results, and we do not enumerate them here.

Two are worth reporting, because they did reach published numbers.

A ground-truth error, found after the first analysis. One cross-document question accepted a single expected parent where the loaded data contains three filers that genuinely list the subsidiary. All three arms were naming a real parent and being scored wrong. Because every model answer is stored, correcting the ground truth and re-scoring cost no additional inference; it moved text-to-SQL from 28 to 29 passing attempts, vector RAG from 6 to 9 and GraphRAG from 44 to 45. That we found it after believing the results were final is direct evidence that others may remain.

An asymmetry that penalised the graph, and what fixing it changed. The more instructive defect is one we initially reported as a finding. Both stores keep a normalised name column, lowercased with corporate suffixes stripped, so *AEP Texas Inc.* is stored as `aep texas`. Neither schema description said so. Text-to-SQL was unaffected because it matched the raw column with `ILIKE`; the graph arm wrote an exact property match against `nameNormalized` using the name as it appeared in the question, retrieved zero rows, and refused. We first published that as a category loss for structural retrieval.

It is not one. It is the same class of defect as an asymmetry we had already found and fixed in the opposite direction — an earlier iteration documented a name convention to the graph but not to SQL, making text-to-SQL fail for a reason unrelated to its architecture. Having recorded that, we should have checked for its mirror image and did not.

We therefore documented the normalisation identically in both schema descriptions, changing no retrieval code and no question, and re-ran all 180 attempts. Table 8 reports both measurements. We present the repaired numbers as the paper’s result and the original alongside them, because a fix that moves a headline in the author’s favour should be visible rather than quietly absorbed.

The general lesson is not that our harness had bugs. It is that a benchmark harness is itself an instrument requiring calibration, and that a list of defects already found is only useful if it is read as a checklist for the ones not yet found.

9.3 Artifacts

The ingestion pipeline, the three retrieval arms, the scoring harness, the question set with its ground truth, and the stored answers for all 180 attempts are retained, which is what made the re-scoring in item 8 possible without new inference. Every table and figure in this paper is generated programmatically from the stored results rather than transcribed, so the numbers in the prose and the numbers in the tables cannot drift apart.

	Text-to-SQL		Vector RAG		GraphRAG	
	pub.	rep.	pub.	rep.	pub.	rep.
Questions solved (of 20)	9	10 ↑	3	–	14	16 ↑
Attempts passed (of 60)	29	36 ↑	9	–	45	50 ↑
Confident falsehoods	2	6 ↑	0	–	0	–
Correct but uncited	16	10 ↓	6	12 ↑	0	–
Total tokens (thousands)	158	144 ↓	119	118 ↓	780	521 ↓
T1 attempts (of 15)	0	7 ↑	3	–	15	–
T2 attempts (of 15)	15	–	0	–	6	–
T3 attempts (of 15)	0	–	0	–	15	14 ↓
T4 attempts (of 15)	14	–	6	–	9	15 ↑

Table 8: The published measurement (*pub.*) and the repaired one (*rep.*) after documenting the name normalisation identically in both schema descriptions. Nothing else changed: same questions, same retrieval code, same model and settings, and the statement timeout added later for the live service was disabled so that exactly one variable differs. Shown in full because the repair moves a headline in the author’s favour.

Resource	Specification	USD/mo
EC2 (app, graph, gateway)	m7g.xlarge, 4 vCPU / 16 GB	85.12
EC2 storage	100 GB gp3	9.12
RDS PostgreSQL	db.t4g.medium	61.32
RDS storage	100 GB gp3	9.12
S3 + CloudFront	demo volume	0.50
NAT gateway	not used (S3 VPC endpoint)	0.00
Total		165

Table 9: Monthly infrastructure cost, on-demand pricing in `ap-south-1`. The embedding pass used a temporarily resized 16-vCPU host at 97 chunks/s; the instance was scaled back afterwards.

10 Conclusion

We measured three retrieval architectures over one corpus and one question set: 167,050 SEC filings from 10,511 companies, 508,714 embedded chunks and a graph of 430,257 nodes and 561,144 edges, queried with 20 questions in four shape categories, each asked three times, for 180 attempts scored programmatically rather than by an LLM. GraphRAG solved 16 of 20 questions on all three attempts, text-to-SQL 10 and vector RAG 3.

The aggregate is the least useful number in the paper. GraphRAG lost structured aggregation outright and expensively, and the defensible summary is a rule about question shape rather than a winner:

- If the answer lies in one document and the entity is named, the binding constraint is *entity filtering*, not ranking. Similarity search alone fails at scale — 3 of 15 here — and needs either a metadata filter on a stable identifier or a traversal to supply one.
- If the question is a `GROUP BY` over typed columns, use SQL. A traversal adds no information, adds a place to go wrong, and cost 12.2 times the tokens to score 6 of 15 against 15 of 15.
- If the question requires resolving *who* an entity is, or joining facts that separate documents assert, structural retrieval is worth its cost: 14 of 15 and 15 of 15 on the two such categories, against 0 of 15 and 14 of 15 for the strongest baseline.
- If provenance must be auditable, the graph’s advantage is categorical rather than marginal. It was never uncited across 60 attempts, because the traversal returns accession numbers before any text is read, whereas the two baselines were right-but-unciteable 10 and 12 times.

The safety finding is narrower but sharper than the accuracy finding, and it is the one we did not anticipate. All six confident falsehoods were text-to-SQL conflating distinct people who share a surname, and the count *tripled* when we gave that baseline a better schema description. Capability and safety moved in opposite directions: knowing the schema converted honest refusals into assertive wrong answers. Resolving identity to an identifier before reading any text prevents that class of error structurally rather than probabilistically. For an application where a confident wrong answer costs more than a refusal, that property may matter more than the pass rate.

We claim none of this beyond one corpus, one model and twenty questions — and one of our own published category results turned out to be a documentation bug rather than a finding, which is the strongest caution we can offer about how much weight a single benchmark run should carry.

Future work

Four steps would materially strengthen these results. *Re-run all 180 attempts per model*, turning the cross-model probe of Table 7 into a measurement and separating architectural effects from model effects. *Repair the entity-resolution layer* — `:RESOLVES_TO` still covers 0.12% of subsidiary nodes —

and establish how much of the T4 result depends on it. *Obtain an independent question set*, since the present one was written by the author of the graph, which remains the study’s most serious threat. And *vary the retriever*: a longer-context encoder with a metadata filter on CIK would test directly whether the single-document collapse is a property of embeddings or of this particular configuration — our own diagnosis predicts the metadata filter, not the encoder, does the work.

References

- [1] U.S. Securities and Exchange Commission. Edgar full-text search and bulk data access. <https://www.sec.gov/edgar>, 2024. Fair-access policy limits automated requests to 10 per second.
- [2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [3] Yu A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018.
- [4] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- [5] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [6] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921*, 2024.
- [7] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.
- [8] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of EMNLP-IJCNLP*, 2019.
- [9] Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103*, 2017.
- [10] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of EMNLP*, 2018.
- [11] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 2021.
- [12] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. In *Proceedings of COLING*, 2020.

- [13] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 2022.
- [14] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation. In *Proceedings of EACL: System Demonstrations*, 2024.
- [15] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 2023.
- [16] Andrew Kane. pgvector: Open-source vector similarity search for postgres. <https://github.com/pgvector/pgvector>, 2024.
- [17] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [18] Neo4j, Inc. Neo4j graph database. <https://neo4j.com>, 2024.

A The question set

Table 10 lists all twenty questions verbatim. Each was asked three times of each of the three architectures, giving the 180 scored attempts analysed in Section 6.

ID	Question
T1-1	What manufacturing and supply chain risks did Moderna, Inc. disclose in the risk factors of its 10-K?
T1-2	What supply chain and food safety risks did Yum China Holdings, Inc. disclose in the risk factors of its 10-K?
T1-3	What cybersecurity and data privacy risks did ROKU, INC disclose in the risk factors of its 10-K?
T1-4	What cybersecurity and competition risks did SentinelOne, Inc. disclose in the risk factors of its 10-K?
T1-5	What regulatory and reimbursement risks did Privia Health Group, Inc. disclose in the risk factors of its 10-K?
T2-1	Which audit firm appears as the auditor for the most companies in this dataset, and how many distinct companies does it audit?
T2-2	Which company discloses the most subsidiaries in its EX-21 exhibit, and how many does it disclose?
T2-3	What is the most common jurisdiction of incorporation among all disclosed subsidiaries, and how many subsidiaries are incorporated there?
T2-4	Which SEC form type appears most often in this dataset, and how many filings of that type are there?
T2-5	Which issuer has the most distinct insiders filing Forms 3, 4 or 5 against it, and how many distinct insiders are there?
T3-1	Brian Wendling holds an insider role at more than one company in this dataset. For every company where they are an officer or director, list that company’s disclosed subsidiaries and summ...
T3-2	Monica Lozano holds an insider role at more than one company in this dataset. For every company where they are an officer or director, list that company’s disclosed subsidiaries and summa...
T3-3	William Giles holds an insider role at more than one company in this dataset. For every company where they are an officer or director, list that company’s disclosed subsidiaries and summa...
T3-4	Judy Bruner holds an insider role at more than one company in this dataset. For every company where they are an officer or director, list that company’s disclosed subsidiaries and summari...
T3-5	Gregg Sengstack holds an insider role at more than one company in this dataset. For every company where they are an officer or director, list that company’s disclosed subsidiaries and sum...
T4-1	Is "American Airlines, Inc." listed as a subsidiary in another company’s EX-21 exhibit while also being an SEC filer in its own right? If so, name the parent that lists it and confirm w...
T4-2	Is "AEP Texas Inc." listed as a subsidiary in another company’s EX-21 exhibit while also being an SEC filer in its own right? If so, name the parent that lists it and confirm whether it...
T4-3	Is "Athene Holding Ltd." listed as a subsidiary in another company’s EX-21 exhibit while also being an SEC filer in its own right? If so, name the parent that lists it and confirm wheth...
T4-4	Is "Alexander’s, Inc." listed as a subsidiary in another company’s EX-21 exhibit while also being an SEC filer in its own right? If so, name the parent that lists it and confirm whether...
T4-5	Is "AllianceBernstein Holding L.P." listed as a subsidiary in another company’s EX-21 exhibit while also being an SEC filer in its own right? If so, name the parent that lists it and co...

Table 10: The twenty benchmark questions. T1 single-document semantic, T2 structured aggregation, T3 multi-hop relational plus text, T4 cross-document entity resolution.